



Concurrency Control

- ▶ Concurrency Control is the working concept that is required for controlling and managing the concurrent execution of database operations and thus avoiding the inconsistencies in the database.
- ▶ Thus, for maintaining the concurrency of the database, we have the concurrency control protocols.

Concurrency Control Protocols

- ▶ The concurrency control protocols ensure the atomicity, consistency, isolation, durability and serializability of the concurrent execution of the database transactions. Therefore, these protocols are categorized as:
 - ▶ Lock Based Concurrency Control Protocol
 - ▶ Time Stamp Concurrency Control Protocol
 - ▶ Validation Based Concurrency Control Protocol
- ▶ We will understand and discuss each protocol one by one in our next sections.

Lock-Based Protocol

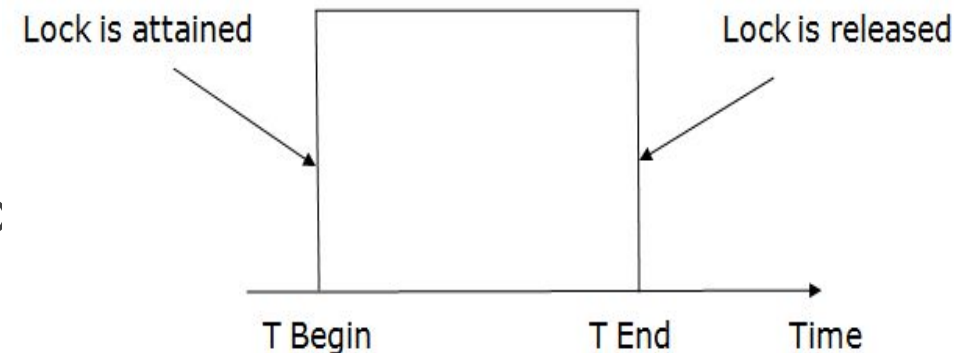
- ▶ In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. There are two types of lock:
- ▶ **1. Shared lock:**
- ▶ It is also known as a Read-only lock. In a shared lock, the data item can only read by the transaction.
- ▶ It can be shared between the transactions because when the transaction holds a lock, then it can't update the data on the data item.
- ▶ **2. Exclusive lock:**
- ▶ In the exclusive lock, the data item can be both reads as well as written by the transaction.
- ▶ This lock is exclusive, and in this lock, multiple transactions do not modify the same data simultaneously.

There are four types of lock protocols available:

- ▶ **1. Simplistic lock protocol**
- ▶ It is the simplest way of locking the data while transaction. Simplistic lock-based protocols allow all the transactions to get the lock on the data before insert or delete or update on it. It will unlock the data item after completing the transaction.

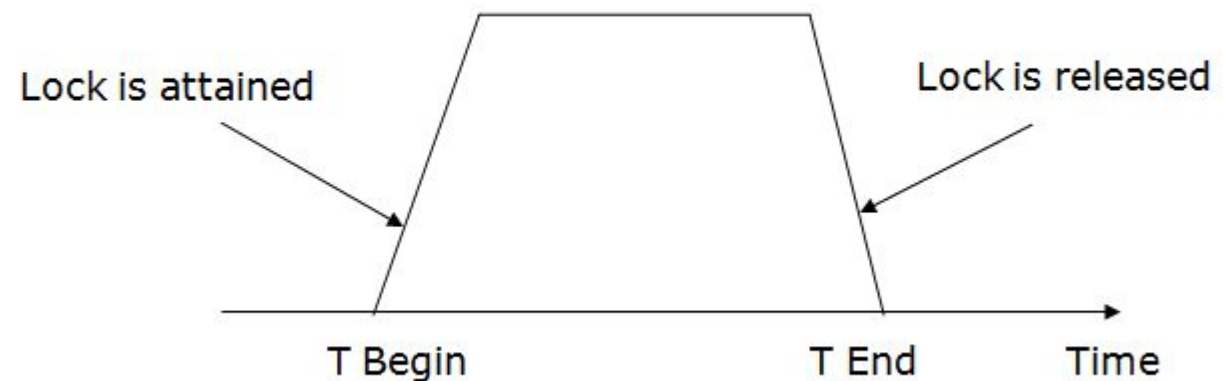
2. Pre-claiming Lock Protocol

- ▶ Pre-claiming Lock Protocols evaluate the transaction to list all the data items on which they need locks.
- ▶ Before initiating an execution of the transaction, it requests DBMS for all the lock on all those data items.
- ▶ If all the locks are granted then this protocol allows the transaction to begin. When the transaction is completed then it releases all the lock.
- ▶ If all the locks are not granted then this
- ▶ protocol allows the transaction to rolls b
- ▶ and waits until all the locks are granted



3. Two-phase locking (2PL)

- ▶ The two-phase locking protocol divides the execution phase of the transaction into three parts.
- ▶ In the first part, when the execution of the transaction starts, it seeks permission for the lock it requires.
- ▶ In the second part, the transaction acquires all the locks. The third phase is started as soon as the transaction releases its first lock.
- ▶ In the third phase, the transaction releases all the locks it has acquired.



There are two phases of 2PL:

- ▶ **Growing phase:** In the growing phase, a new lock on the data item may be acquired by the transaction, but none can be released.
- ▶ **Shrinking phase:** In the shrinking phase, existing lock held by the transaction may be released, but no new locks can be acquired.
- ▶ In the below example, if lock conversion is allowed then the following phase can happen:
- ▶ Upgrading of lock (from $S(a)$ to $X(a)$) is allowed in growing phase.
- ▶ Downgrading of lock (from $X(a)$ to $S(a)$) must be done in shrinking phase.

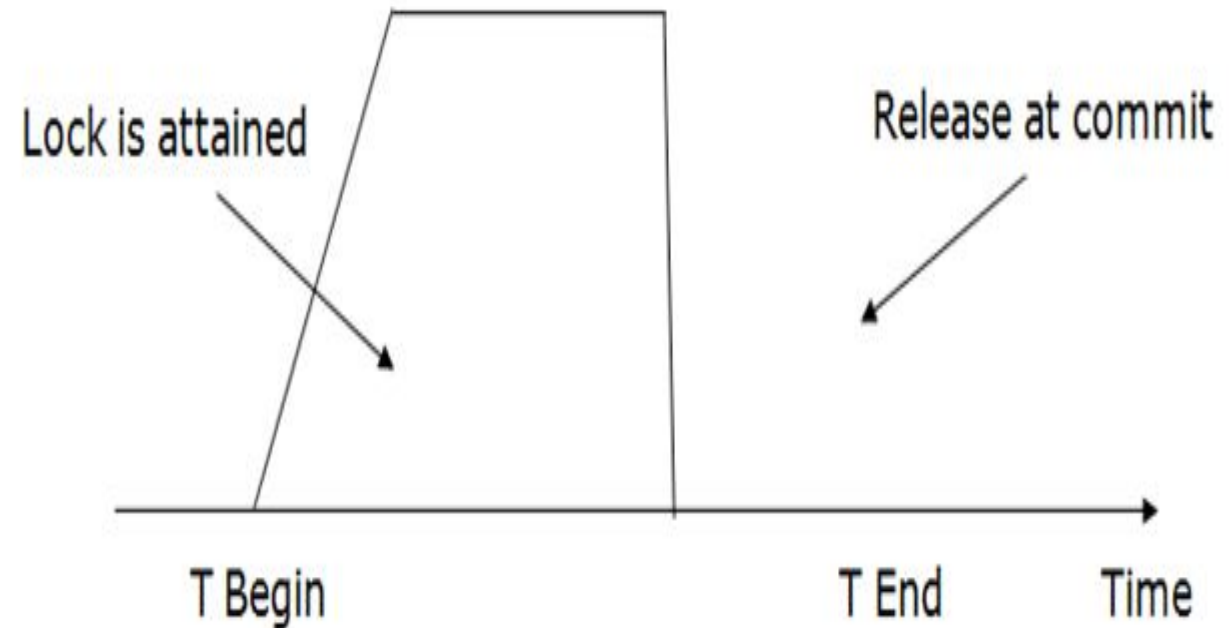
two phases of 2PL

- ▶ **Transaction T1:**
- ▶ **Growing phase:** from step 1-3
- ▶ **Shrinking phase:** from step 5-7
- ▶ **Lock point:** at 3
- ▶ **Transaction T2:**
- ▶ **Growing phase:** from step 2-6
- ▶ **Shrinking phase:** from step 8-9
- ▶ **Lock point:** at 6

	T1	T2
0	LOCK-S(A)	
1		LOCK-S(A)
2	LOCK-X(B)	
3	—	—
4	UNLOCK(A)	
5		LOCK-X(C)
6	UNLOCK(B)	
7		UNLOCK(A)
8		UNLOCK(C)
9	—	—

4. Strict Two-phase locking (Strict-2PL)

- ▶ The first phase of Strict-2PL is similar to 2PL. In the first phase, after acquiring all the locks, the transaction continues to execute normally.
- ▶ The only difference between 2PL and strict 2PL is that Strict-2PL does not release a lock after using it.
- ▶ Strict-2PL waits until the whole transaction to commit, and then it releases all the locks at a time.
- ▶ Strict-2PL protocol does not have shrinking phase of lock release.

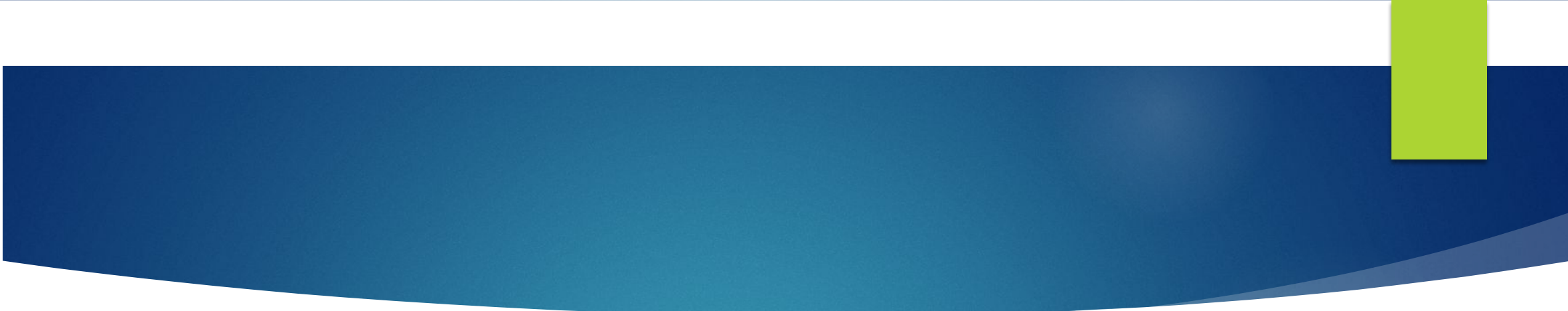


It does not have cascading abort as 2PL does.

Time Stamping

- ▶ The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is nothing but the ascending order of the transaction creation.

The priority of the older transaction is higher that's why it executes first. To determine the timestamp of the transaction, this protocol uses system time or logical counter.

- 
- ▶ The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time. But Timestamp based protocols start working as soon as a transaction is created.
 - ▶ Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.
 - ▶ The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

Basic Timestamp ordering protocol works as follows:

- ▶ 1. Check the following condition whenever a transaction T_i issues a **Read (X)** operation:
 - ▶ If $W_TS(X) > TS(T_i)$ then the operation is rejected.
 - ▶ If $W_TS(X) \leq TS(T_i)$ then the operation is executed.
 - ▶ Timestamps of all the data items are updated.
- ▶ 2. Check the following condition whenever a transaction T_i issues a **Write(X)** operation:
 - ▶ If $TS(T_i) < R_TS(X)$ then the operation is rejected.
 - ▶ If $TS(T_i) < W_TS(X)$ then the operation is rejected and T_i is rolled back otherwise the operation is executed.

Advantages and Disadvantages of TO protocol:

- ▶ **TS(Ti)** denotes the timestamp of the transaction Ti.
- ▶ **R_TS(X)** denotes the Read time-stamp of data-item X.
- ▶ **W_TS(X)** denotes the Write time-stamp of data-item X.
- ▶ **Advantages and Disadvantages of TO protocol:**
- ▶ TO protocol ensures serializability since the precedence graph is as follows:
- ▶ TS protocol ensures freedom from deadlock that means no transaction ever waits.
- ▶ But the schedule may not
- ▶ be recoverable and may
- ▶ not even be cascade-free.



Image: Precedence Graph for TS ordering



All the Best